Lab 00 — The Linux fundamentals Docker assumes you know

Theory — the kernel, processes, users, files, the shell, and just enough networking. The labs that follow rely on all of it without explaining it again.

Objectives

- Know the three layers of a Linux system: kernel, system calls, userland.
- Describe a process: PID, parent, environment, signals, exit code.
- Read a line of `ls -l` — owner, group, permissions — and know what `root` can do that you can't.
- Use the shell effectively: environment variables, `PATH`, redirections, pipes.
- See where each of these ideas shows up again in the Docker labs.

---

1. Why a Linux lab in a Docker course

Because a container is **nothing but Linux**. In lab 01 you will read that "a container is an isolated process". That sentence only helps if you have a precise idea of what a process is: something with a number, a parent, an environment, and a way of dying. The same goes for the rest: Podman's "rootless" mode makes no sense without UIDs, a port that is "already in use" makes no sense without ports, and volumes make no sense without mounts. This lab teaches exactly that vocabulary — nothing more — and each section points to the lab where the idea comes back.

> **→ HISTORY**
> Unix appeared in 1969 at Bell Labs and established two ideas that still shape everything: *everything is a file*, and *small programs combined together*. In 1991, a Finnish student named Linus Torvalds wrote a free Unix-compatible kernel: Linux. Ubuntu (2004) is a **distribution** of it — the Linux kernel plus a curated, packaged userland. And since 2019, Microsoft has shipped its own Linux kernel inside Windows. That is WSL 2, the machine you will use for these labs.

2. The kernel and the userland

A Linux system has two layers. At the bottom sits the **kernel**, the only program that touches the hardware. It creates processes, hands out CPU time and memory, reads disks, and sends network packets. Everything else — `bash`, `ls`, `java`, your application — runs on top and is called the **userland**. A single boundary separates the two: **system calls** (*syscalls*). A program never reads a file on its own; it asks the kernel to `open` and then `read`, and the kernel decides whether to allow it.

This boundary explains two things you will see constantly. First, portability: a Linux binary runs on any distribution, because all it ever uses are system calls, and those are identical everywhere. Second, control: since everything goes through the kernel, the kernel only has to lie a little ("you are process 1", "this is your `/`") to isolate a process — which is exactly what a container does, as you will see in lab 01.

→ WINDOWS / WSL
A Linux program can only talk to a Linux kernel; the Windows kernel speaks a different, incompatible language. **WSL 2** (*Windows Subsystem for Linux*) solves this by running a real Linux kernel inside a tiny VM managed by Windows. Your Ubuntu 24.04 lives inside that VM: `uname -r` there reports `...microsoft-standard-WSL2`, the signature of the kernel Microsoft builds. The Windows drive shows up under `/mnt/c`.

3. Processes

A **process** is a running program: code, memory, and an identity. The kernel assigns it a unique **PID** (*process ID*) and records its parent, the **PPID**. Every process is created by another one — when you type `ls`, your shell clones itself (`fork`) and the clone replaces itself with `ls` (`exec`). At boot, the kernel starts one first process, **PID 1** (`systemd` on Ubuntu), the ancestor of all others. When a parent dies before its child, PID 1 adopts the orphan. Keep that number in mind: inside a container, *your application* becomes PID 1, which brings surprising responsibilities (lab 03).

→ LINUX
A **daemon** is a service process. Started at boot by `systemd` and detached from any terminal, it runs in the background until it is needed: `sshd` waits for SSH connections, `cron` waits for the next scheduled job. By convention its name ends in "d". Remember the word — Docker is built around a daemon called `dockerd`, and Podman's defining feature is that it has none. That debate fills lab 01.

Every process ends by returning an **exit code**: `0` means success, anything else means failure. The shell keeps it in `$?`. A few conventional values: `1` general error, `2` wrong usage, `126` file not executable, `127` command not found, `128 + n` killed by signal *n*.

You never "close" a process — you send it a **signal**, a numbered notification delivered by the kernel. Three signals matter here:

Signal	Number	Meaning	Can the process ignore it?
`SIGTERM`	15	"Shut down cleanly"	Yes — it gets a chance to save, close, clean up
`SIGKILL`	9	Immediate death, enforced by the kernel	**No** — and nothing gets cleaned up
`SIGINT`	2	Keyboard interrupt (`Ctrl+C`)	Yes

REMEMBER
`kill` doesn't actually mean "kill"; it means "send a signal", and by default it sends the polite `SIGTERM`. A process killed with `SIGKILL` exits with code `137` (128 + 9). You will keep running into that number for as long as you work with Docker: it is the signature of a container that was stopped by force — often because it ran out of memory.

The kernel publishes the state of every process in `/proc`, a directory that isn't really one: `/proc/1234/` describes process 1234 — its command line, its environment, its limits — generated on the fly and taking up no disk space at all. `ps` does nothing more than read it.

4. Users, groups, permissions

Every process runs *as* someone: a **user**, identified by a number called the **UID**, plus one or more **groups** (GID). The kernel only ever deals in numbers; the names (`kevin`, `postgres`) come from the file `/etc/passwd`. The first user on an Ubuntu system gets UID **1000**. The user `root`, UID **0**, is

special: the kernel denies it nothing. The `sudo` command runs a single command *as* root and logs who asked for it.

Every file has an owner, a group, and nine permission bits, all visible in `ls -l`:

```
-rw-r----- 1 root shadow 1234 ... /etc/shadow
 └┬┘└┬┘└┬┘    └──┬──┘ owner root, group shadow
  │  │  └ others: nothing
  │  └ group shadow: read
  └ root: read + write
```

`r` means read, `w` write, `x` execute (for a directory: enter it). `chmod` changes the bits, `chown` changes the owner. One detail trips everyone up: a script must be **executable** (`chmod +x`) before `./script.sh` will run it. Otherwise the shell answers `Permission denied` with exit code 126.

→ SECURITY
The golden rule: never work as root; use `sudo` to elevate only when a specific command needs it. This is the Linux version of the principle of least privilege, and it is the core argument for **rootless** Podman: your containers will run under UID 1000, not UID 0, so a compromised application gets your rights and nothing more (lab 01).

PITFALL
The kernel compares **numbers**, not names. A file created by UID 1000 inside a container belongs to UID 1000 everywhere, even when the displayed name differs from one system to the next. Obvious as it sounds, this fact becomes the classic volume-permissions headache of lab 06.

5. Files, the tree, mounts

Under Unix, *everything is a file*: documents, but also disks (`/dev/sdc`), the kernel's state (`/proc`), even sockets. There are no `C:` or `D:` drives. There is one tree, rooted at `/`, and every disk or filesystem is **mounted** — attached — onto some directory in it. `findmnt /` tells you which disk provides the root; on WSL, `/mnt/c` is where the Windows drive is mounted. Mounting, unmounting, and stacking filesystems is precisely how Docker images and volumes work under the hood (labs 02 and 06).

The standard directories worth recognizing: `/etc` (configuration), `/home` (your files), `/usr/bin` (programs), `/var` (data that grows: logs, databases), `/tmp` (scratch space), `/proc` and `/sys` (windows into the kernel).

6. The shell: environment, PATH, pipes

The **shell** (`bash`) is an ordinary process whose job is to start other processes. Three of its mechanisms are essential Docker knowledge.

Environment variables. Every process starts life with a key=value dictionary inherited from its parent: `HOME`, `PATH`, `LANG`, and so on. A plain shell variable (`MSG=hello`) stays local; it only enters the environment of child processes once you run `export MSG`. This is how containers get configured — in lab 08, your Spring Boot application will read its database password from an environment variable, never from a file baked into the image.

→ JAVA
A JVM is an ordinary process: `java -jar app.jar` has a PID, a UID, and an environment. Spring Boot reads that environment at startup, so setting `SERVER_PORT=9090` changes its port without touching the JAR. Calling `System.getenv("HOME")` in Java reads the same inherited dictionary.

The `PATH`. When you type `ls`, the shell searches the directories listed in the `PATH` variable, in order, for an executable named `ls`. `which ls` shows what it found; `command not found` (code 127) means the search came up empty. This is also why you run a script from the current directory as `./script.sh` — the current directory is deliberately left out of the `PATH`.

Redirections and pipes. A process has three streams: input (0, *stdin*), output (1, *stdout*) and errors (2, *stderr*). The shell can plug them in anywhere: `> file` redirects output, `2>` redirects errors, `2>&1` merges the two, and `command1 | command2` feeds one command's output into the next one's input. You will build such pipelines in every lab (`podman ps | grep …`), and a container's logs are simply its streams 1 and 2, captured (lab 03).

7. Just enough networking

Three ideas will carry you through to lab 07. **The interface**: a machine's network socket, holding an IP address. `lo`, the *loopback* interface, holds `127.0.0.1`, better known as `localhost` — the machine talking to itself. **The port**: a number from 1 to 65535 that tells services on the same address apart. Only one process can listen on a given port, and `ss -tlnp` shows who is listening where. **The privilege rule**: ports below 1024 are reserved for root. That is why your test server will listen on 8080 rather than 80, and why rootless Podman refuses `-p 80:80` (lab 07).

→ NETWORK
`curl` is the Swiss army knife here: it sends an HTTP request and prints the raw response. `curl -i http://localhost:8080/` shows the status code (`200 OK`, `404` …), the headers, and the body. It is the go-to tool for testing a containerized API without a browser.

→ WINDOWS / WSL
WSL 2 forwards `localhost` automatically: a server listening on port 8080 *inside* Ubuntu can be reached from a **Windows** browser at `http://localhost:8080`. Convenient — just remember that this forwarding is a courtesy of WSL, not something Linux does by itself.

8. In the workplace

The whole container ecosystem industrializes these ideas. A production Spring Boot server boils down to: a `java` process (PID) started by an unprivileged application user (UID), configured through environment variables, writing its logs to *stdout*, listening on port 8080, and stopped with `SIGTERM` at every deployment. An operator investigating an incident runs `ps`, `ss` and `curl`, checks `$?`, and digs through logs with `grep`. Docker replaces none of this — it packages it.

PODMAN
Podman takes the same logic to its conclusion: no daemon, just *your* user (UID 1000) starting processes. Everything in this lab — UIDs, signals, `/proc`, unprivileged ports — describes exactly what Podman can do without `sudo`. Docker, in contrast, depends on a daemon that runs as root (`dockerd`). That difference is the subject of lab 01.

Remember

- The **kernel** controls everything; programs can only make **system calls**. Isolating a process means making the kernel lie to it — the founding idea behind containers.
- A **process** has a PID, a parent, and an inherited environment, and it ends with an **exit code**: `0` = success, `137` = killed by SIGKILL.
- `SIGTERM` asks; `SIGKILL` enforces. A well-behaved service shuts down on SIGTERM.
- The kernel thinks in numeric **UID/GID**; `root` = UID 0 = unlimited rights; `sudo` grants them one command at a time.
- There is one file tree; disks and virtual filesystems are **mounted** into it; `/proc` is your window into the kernel.
- **Environment variables** flow from parent to child; the `PATH` decides which commands exist.
- A service = an address + a **port**; `localhost` = this machine; ports below 1024 belong to root.

Vocabulary

kernel: the program that controls hardware and processes. — **userland**: everything that runs on top of the kernel. — **system call**: a program's request to the kernel (`open`, `fork`…). — **process**: a running program, identified by a **PID**. — **PID 1**: the first process, ancestor and guardian of all others. — **daemon**: a background service process, without a terminal, managed by `systemd` (`sshd`, `dockerd`). — **signal**: a notification sent to a process (`SIGTERM`, `SIGKILL`). — **exit code**: the integer a process returns when it dies; `0` = success; stored in `$?`. — **UID / GID**: user and group numbers, the only identities the kernel understands. — **root**: UID 0, exempt from every permission check. — **mount**: attaching a filesystem to a directory in the tree. — **/proc**: a virtual tree exposing the state of the kernel and of every process. — **environment variable**: a key=value pair inherited by child processes. — **PATH**: the list of directories the shell searches for commands. — **stdin / stdout / stderr**: the three standard streams (0, 1, 2). — **pipe**: connecting one process's output to another's input. — **port**: a number identifying a service on an IP address. — **localhost**: `127.0.0.1`, the machine's own address.